# Lecture: Classification: Ex, Digits Dataset

Author: Refat Othman

## 1 What is the Digits Dataset?

The **Digits dataset** is a built-in dataset in scikit-learn used for **multi-class classification**.

It represents **handwritten digits (0–9)**.

Each sample is:

- An 8 × 8 grayscale image
- Converted into 64 numerical features
- Pixel intensity values range from 0 to 16

Dataset Properties:

- Total samples: 1797
- Features per sample: 64
- Classes: 10 (digits 0 → 9)
- Balanced dataset (approximately 180 samples per class)

## 2 Loading the Dataset

```
from sklearn.datasets import load_digits
import numpy as np

digits = load_digits()

X = digits.data      # Shape (1797, 64)
y = digits.target    # Shape (1797,)

print("X shape:", X.shape)
print("Number of classes:", len(np.unique(y)))
```

Explanation:

- Each row in X = one image
- Each column = one pixel
- y contains the correct digit label

# 3 Visualizing Images

```
import matplotlib.pyplot as plt

plt.imshow(digits.images[0], cmap='gray')
plt.title(f"Label: {digits.target[0]}")
plt.axis('off')
plt.show()
```

🧠 Teaching Point:

- images shape = (1797, 8, 8)
- data shape = (1797, 64)
- The image is flattened before classification

---

# 4 Train/Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,                # X = features (images as 64 numbers), y = labels (digits
0-9)
    test_size=0.2,       # 20% of the data will be used for testing
    random_state=42,     # ensures reproducibility (same split every run)
    stratify=y           # keeps same class distribution in train and test
)
```

🔎 Detailed Explanation Line by Line:

1 `train_test_split`:

- A function that randomly splits the dataset into two parts:
  - Training set → used to train the model
  - Test set → used to evaluate the model

2 `X, y`:

- X contains the input features (1797 × 64 matrix)
- y contains the labels (1797 digits from 0 to 9)
- Both must be split in the same way to keep correct pairing

3 `test_size=0.2`:

- 20% of the dataset goes to testing

- Since we have 1797 samples:
    - Test ≈ 360 samples
    - Train ≈ 1437 samples

4️⃣ `random_state=42`:

- Fixes the random seed
- Ensures that every time we run the code, we get the SAME split
- Important for reproducibility in research and teaching

5️⃣ `stratify=y`:

- Ensures that the proportion of each digit (0–9) is preserved
- Example:
    - If digit "3" represents 10% of the full dataset
    - It will also represent ~10% in train and ~10% in test

⚠️ Why is stratification important?

- Without it, some digits might be underrepresented in test set
- This can bias performance evaluation

📊 After this line we now have:

- X_train → used for learning
- y_train → correct labels for training
- X_test → unseen data
- y_test → true labels for evaluation

Why stratify?

- Keeps equal distribution of digits in train and test

---

# 5️⃣ Scaling Features

Why scaling?

- Pixel values differ in magnitude
- Distance-based models (KNN, SVM) need normalization

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()        # Create a scaler object
X_train = scaler.fit_transform(X_train)  # Learn mean & std from training data,
then scale it
X_test = scaler.transform(X_test)        # Apply SAME scaling parameters to test
data
```

🔬 Deep Explanation of Scaling

What does StandardScaler actually do?

For each feature (each pixel column), it computes:

```
mean = average value of that feature (computed from training data)
std  = standard deviation of that feature
```

Then it transforms every value using:

```
X_scaled = (X - mean) / std
```

This means:

- Each feature will have mean ≈ 0
- Each feature will have standard deviation ≈ 1

📊 Why do we need scaling?

1️⃣ Distance-Based Models (KNN, SVM, Logistic Regression) These models depend on distances or dot products. If one feature has larger numeric range, it dominates the calculation.

Example: If one pixel ranges 0–16 and another ranges 0–1000, the larger one controls the distance completely.

Scaling ensures: All features contribute equally.

2️⃣ Faster Convergence (Logistic Regression, SVM) Gradient-based optimization converges faster when features are centered around 0.

3️⃣ Numerical Stability Large values can cause unstable optimization behavior. Scaling prevents that.

🚫 When is scaling NOT necessary?

Decision Trees and Random Forests:

- They split based on feature thresholds.
- They do NOT use distance.
- Therefore scaling does not affect them significantly.

⚠️ Why fit only on training data?

If we compute mean/std using full dataset, the test data would influence the transformation.

This is called Data Leakage.

Correct procedure:

1. Compute mean & std from training data only.

2. Apply same transformation to test data.

This simulates real-world deployment: In production, we never know future data statistics.

🧠 Teaching Summary:

- Scaling = normalization of feature magnitude.
- Required for distance-based & gradient-based models.
- Not required for tree-based models.
- Always fit on training data only.

---

# 6 Training Multiple Classifiers

We will compare:

- KNN
- SVM (RBF)
- Logistic Regression
- Decision Tree

---

## ◇ KNN

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
```

Concept:

- Finds nearest images
- Uses majority vote

---

## ◇ SVM (RBF Kernel)

```
from sklearn.svm import SVC

svm = SVC(kernel="rbf", C=5.0, gamma="scale")
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
```

Concept:

- Finds optimal decision boundary

- RBF allows nonlinear separation

---

### ◇ Logistic Regression

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(max_iter=4000)
logreg.fit(X_train, y_train)
y_pred_log = logreg.predict(X_test)
```

Concept:

- Linear classifier
- Uses Softmax for multi-class

---

### ◇ Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(max_depth=10, random_state=42)
tree.fit(X_train, y_train)
y_pred_tree = tree.predict(X_test)
```

Concept:

- Creates rule-based splits
- Easy to interpret
- Can overfit

---

## 7 Performance Evaluation

### Accuracy

```
from sklearn.metrics import accuracy_score

print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("LogReg Accuracy:", accuracy_score(y_test, y_pred_log))
print("Tree Accuracy:", accuracy_score(y_test, y_pred_tree))
```

Accuracy = Correct Predictions / Total Samples

---

# 8 Confusion Matrix

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred_svm)
plt.show()
```

📊 What is Confusion Matrix?

A Confusion Matrix is a detailed performance table that shows **exactly how the model is making mistakes**.

It is a square matrix of size (number_of_classes × number_of_classes). Since Digits has 10 classes → the matrix is 10 × 10.

Structure:

- Rows = True Labels (actual digit)
- Columns = Predicted Labels (model prediction)

Each cell (i, j) means: "How many times digit i was predicted as digit j"

✔ Diagonal values (top-left to bottom-right):

- Correct predictions
- These represent accurate classifications

❌ Off-diagonal values:

- Mistakes
- These show which digits are being confused

---

🔢 Simple Numeric Example (3-class case for clarity)

Suppose we had only digits {0, 1, 2} and the confusion matrix is:

```
          Predicted
            0   1   2
True 0     45   3   2
True 1      4  40   6
True 2      1   5  44
```

Interpretation:

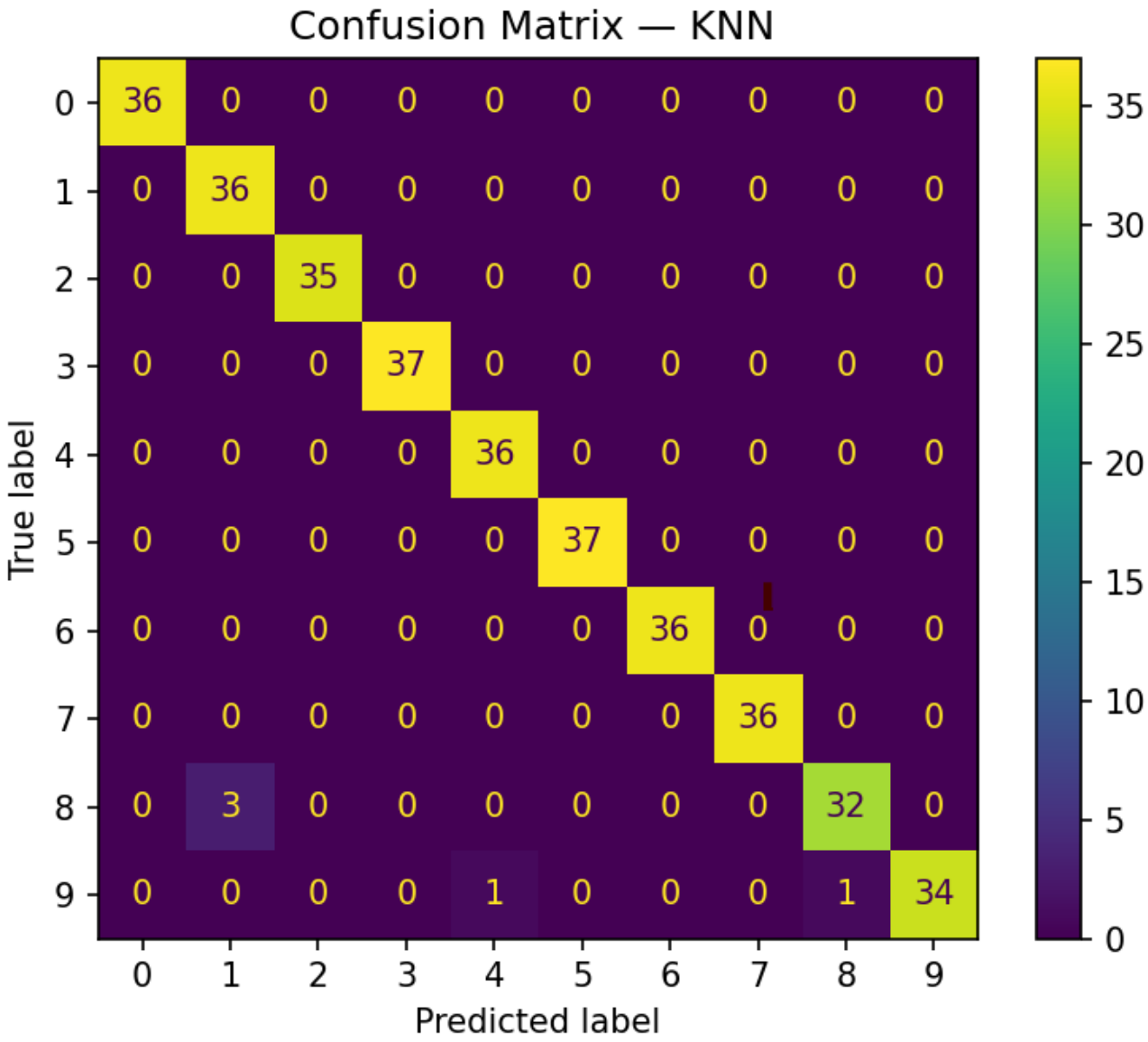
- Digit 0:
    - 45 correctly predicted as 0
    - 3 incorrectly predicted as 1

    - 2 incorrectly predicted as 2

- Digit 1:
    - 40 correct
    - 4 predicted as 0
    - 6 predicted as 2
- Digit 2:
    - 44 correct
    - 1 predicted as 0
    - 5 predicted as 1

So if we see large numbers outside the diagonal between two digits, that means the model confuses those digits.

---




# 🔎 Connecting Confusion Matrix to TP, TN, FP, FN

In **binary classification**, we define:

- **TP (True Positive)** → Model correctly predicts Positive
- **TN (True Negative)** → Model correctly predicts Negative
- **FP (False Positive)** → Model predicts Positive but it is actually Negative
- **FN (False Negative)** → Model predicts Negative but it is actually Positive

But Digits is **multi-class (10 classes)**. So how do we define TP, FP, FN, TN here?

We use a strategy called:

👉 **One-vs-Rest (OvR)**

That means: We treat ONE digit as "Positive" and all other digits as "Negative".

---

## Example: Treat Digit "0" as Positive

From the 3×3 example:

```
          Predicted
            0   1   2
True 0   45   3   2
True 1    4  40   6
True 2    1   5  44
```

For Digit 0:

- **TP (True Positive)** = 45 (True 0 predicted as 0)
- **FN (False Negative)** = 3 + 2 = 5 (True 0 predicted as 1 or 2)
- **FP (False Positive)** = 4 + 1 = 5 (True 1 or 2 predicted as 0)
- **TN (True Negative)** = All remaining correct non-0 predictions = 40 + 6 + 5 + 44 = 95

---

## What does each one mean intuitively?

TP → Model correctly recognized digit 0.

FN → Model missed digit 0 and classified it as something else.

FP → Model falsely claimed a digit was 0 when it wasn't.

TN → Model correctly identified other digits as not 0.

---

## Why is this important?

From these values we compute:

Precision = TP / (TP + FP) Recall = TP / (TP + FN) F1-score = 2 × (Precision × Recall) / (Precision + Recall)

This is exactly what `classification_report()` calculates for each class.

---

🧠 Teaching Insight:

In multi-class problems:

- Every class gets its own TP, FP, FN, TN calculation.
- Metrics are computed per class.
- Then averaged (macro / weighted average).

This connects the confusion matrix directly to Precision, Recall, and F1-score.

---

🔍 Real Digits Dataset Interpretation

If in the real 10×10 matrix we see:

- Many values in row 8, column 3 → The model often predicts 8 as 3.

This tells us:

- The shapes of 8 and 3 are visually similar
- The model struggles separating them

---

🎯 Why Confusion Matrix is Powerful for Teaching

Accuracy only tells us "how many were correct overall". Confusion Matrix tells us:

- Which digits are easy
- Which digits are difficult
- Where the model fails
- Whether errors are random or systematic

This helps students move from "model score" thinking to "model behavior" understanding.

---

# 9 Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_svm))
```

Metrics explained:

Precision:

Definition: Precision measures how accurate the positive predictions are.

Formula: Precision = TP / (TP + FP)

Meaning in Digits example (digit 5):

- Of all samples predicted as "5"
- How many were actually real 5?

High Precision means: → When the model says "this is 5", it is usually correct.

---

Recall:

Definition: Recall measures how many real positive cases were correctly detected.

Formula: Recall = TP / (TP + FN)

Meaning in Digits example (digit 5):

- Of all real digit 5 images in the dataset
- How many did the model successfully find?

High Recall means: → The model does not miss many real 5s.

---

F1-score:

Definition: F1-score combines Precision and Recall into one number.

Formula: F1 = 2 × (Precision × Recall) / (Precision + Recall)

Why not just average them? Because F1 uses the harmonic mean, which penalizes imbalance. If either Precision or Recall is low → F1 becomes low.

F1 is useful when:

- We care about balance between missing cases and false alarms.

# 🔟 Typical Results (Teaching Insight)

Usually:

- SVM → Highest accuracy (~98%)
- Logistic Regression → Very strong (~96-97%)
- KNN → Strong (~95-97%)
- Decision Tree → Lower (~85-90%)

Why?

- SVM handles high-dimensional space well
- Tree overfits pixel noise

---

# 1️⃣1️⃣ Common Student Questions

Q: Why do we scale pixels? A: Because distance-based models depend on magnitude.

Q: Why does Decision Tree perform worse? A: Because it splits per feature independently and cannot capture smooth boundaries well.

---